

Enterprise ERP Development Plan

React + Express + PostgreSQL

Tenant Application and Separate Super Admin Platform

Prepared for	Acompworld Technosoft Pvt. Ltd.
Prepared by	OpenAI - structured development planning draft
Delivery scope	Architecture, phased execution, database chart, standards, governance
Primary stack	React, Express (Node.js), TypeScript, PostgreSQL, Redis, Docker
Reference basis	ERP-Application-Playbook-v2.pdf

This document provides a practical industry-standard implementation plan for building a multi-tenant ERP platform using React, Express, and PostgreSQL. It includes the tenant-facing product, the separate super admin application, the proposed data model, platform standards, security controls, delivery phases, and engineering governance needed to ship and scale the system responsibly.

1. Executive summary

The recommended solution is a modular SaaS ERP platform with a shared application core, strict tenant isolation, and a separate super admin project for platform governance. The tenant application handles business operations such as identity, CRM, sales, projects, invoices, subscriptions, payments, finance, support, and reporting. The super admin platform is intentionally isolated to reduce risk and to keep cross-tenant operations, provisioning, subscription management, support tooling, and platform analytics outside the tenant runtime.

Key architectural decisions

Frontend: React + TypeScript with feature-based modules, React Query, form validation, route guards, and reusable design system components.

Backend: Express + TypeScript with layered architecture (route -> controller -> service -> repository), OpenAPI documentation, validation middleware, and centralized error handling.

Database: PostgreSQL as the source of truth for transactional data, with per-record tenant_id, audit fields, optimized indexes, and row-level isolation patterns.

Platform split: tenant product and super admin product run as separate web projects, separate API surface, separate RBAC model, and separate deployment pipeline.

Operations: Dockerized services, CI/CD, structured logging, monitoring, alerting, migration discipline, backups, and disaster recovery drills.

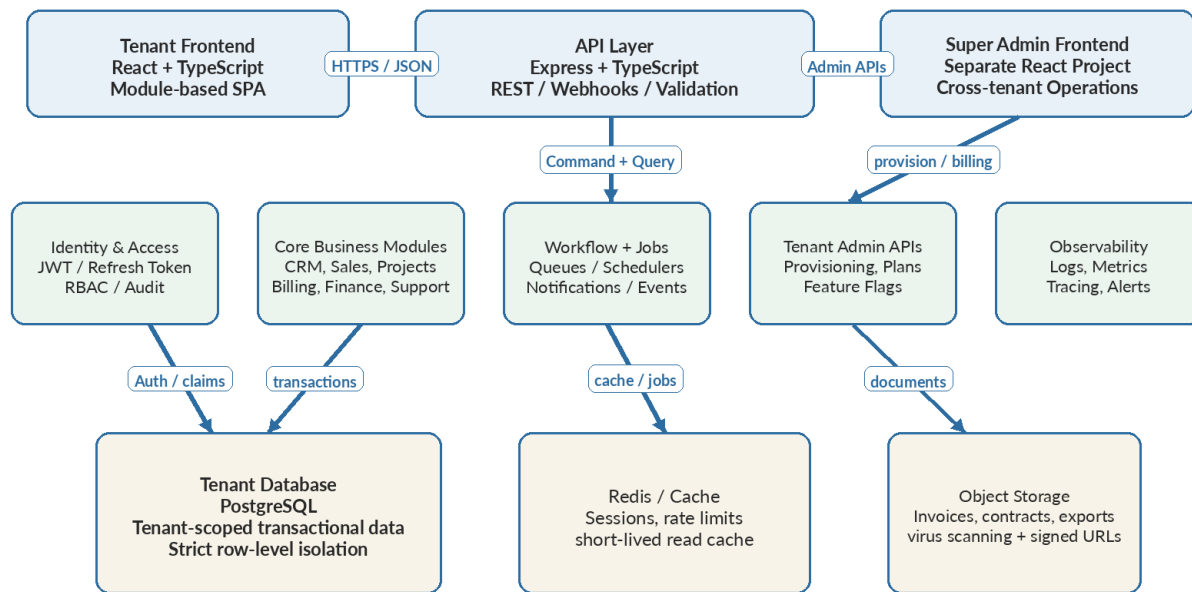
2. Business and technical objectives

- Support multiple tenants on a shared SaaS platform without cross-tenant data exposure.
- Enable rapid delivery of core ERP modules while preserving maintainability and auditability.
- Provide a foundation for subscriptions, invoicing, payments, workflows, reporting, and integrations.
- Make the platform easy to test, deploy, scale, monitor, and extend with additional modules later.
- Keep super admin capability operationally powerful but logically isolated from tenant operations.

3. Recommended solution overview

The preferred implementation model is a modular monolith for phase 1 and phase 2, designed with clear domain boundaries and event-ready contracts so that high-load modules can be extracted into separate services later if the product and team scale justify it. This is the most practical path for a new ERP because it lowers operational complexity while still following the discipline required for long-term enterprise growth.

Target Architecture - ERP Tenant App + Super Admin Platform



Deployment baseline: Docker + CI/CD + PostgreSQL backups + centralized observability. Super admin is isolated as a separate web project with its own routes, roles, and audit trail.

Figure 1. Recommended target architecture for the tenant ERP application and separate super admin platform.

4. Technology stack and standards

Layer	Recommendation	Rationale	Notes
Frontend	React + TypeScript	Large talent pool, reusable components, SPA ergonomics	Prefer Vite, React Router, React Query, Zod, Zustand or Redux Toolkit only where needed
Backend	Node.js + Express + TypeScript	Simple, flexible, fast to staff, good ecosystem	Use strict typing, class-validator or Zod, pino logging, OpenAPI, rate limiting
Database	PostgreSQL	Reliable ACID transactions, indexing, JSONB, reporting capability	Prefer UUID PKs, migration-based schema changes, partition audit logs if needed
Caching / jobs	Redis	Fast cache, queues, throttling	Use for BullMQ job queues, sessions, token blocklist, short-lived projections
Storage	Object storage	Document and report storage	Signed URLs, malware scanning, retention rules

Layer	Recommendation	Rationale	Notes
Authentication	JWT access + refresh token flow	Scalable and API friendly	Short TTL access token, rotation, device/session tracking
DevOps	Docker + CI/CD + IaC	Consistent delivery and recoverability	Separate dev, QA, staging, production environments

5. Application architecture

5.1 Frontend architecture

The React application should use a feature-first folder structure so each business domain owns its pages, components, hooks, API client adapters, validation schema, and tests. Cross-cutting concerns such as design tokens, layout shells, route guards, tables, modals, and notifications should live in shared UI packages or app-level libraries rather than inside feature folders.

- Use TypeScript strict mode, path aliases, ESLint, Prettier, and import boundaries.
- Keep business rules out of UI components; move them into domain services or shared client-side mappers only when necessary for display logic.
- Use React Query for server state, request caching, retry control, optimistic updates, and invalidation.
- Use form libraries with schema validation for complex workflows such as invoice creation, subscription changes, and approval forms.
- Build accessibility, localization readiness, and responsive behavior from the start instead of treating them as later fixes.

5.2 Backend architecture

The Express backend should be organized by domain modules. Each module exposes routes, controllers, services, repositories, DTOs, validators, and tests. Controllers stay thin. Services hold application rules. Repositories encapsulate SQL or query builder logic. Transactions should be coordinated at the service layer. Cross-module calls should happen through service contracts, emitted domain events, or internal command interfaces rather than direct table access.

- Middleware chain: request ID -> auth -> tenant context -> validation -> controller -> service -> structured response.
- Standard error model with domain errors, validation errors, forbidden errors, not-found errors, and safe user-facing messages.
- Idempotency keys for financial endpoints such as payment initiation, webhook handling, invoice issue/post actions, and exports.
- Version APIs from day one, even if v1 is the only version at launch.
- Use OpenAPI generation and publish internal API documentation continuously.

5.3 Suggested domain modules

Module	Primary responsibility	Owns key entities	Integration notes
Identity & RBAC	Authentication, sessions, tenant-aware authorization	users, roles, permissions, sessions	Shared by tenant app and admin app through separate API policies

Module	Primary responsibility	Owns key entities	Integration notes
Tenant setup	Tenant profile, plan, settings, feature flags	tenants, tenant_settings, plan_assignments	Provisioned from super admin
CRM	Customers, contacts, addresses	customers, contacts, customer_addresses	Feeds projects, invoices, subscriptions, support
Sales	Leads, opportunities, quotations	leads, opportunities, quotations	Converts into customers/projects/invoices
Projects	Delivery planning and billable work	projects, milestones, project_members	Can trigger billing events
Billing	Invoices, credit notes, tax snapshots	invoices, invoice_items, credit_notes	Consumes customer and subscription data
Subscriptions	Recurring plans and renewals	subscription_plans, subscriptions, subscription_changes	Generates billing schedules
Payments	Payment intent, reconciliation, refunds	payments, payment_attempts, refunds	Integrates with gateway via webhooks
Finance	Ledger, journal entries, fiscal periods	journals, ledger_entries, accounts	Consumes billing and payment events
Workflow	Approvals, policies, SLA/rules	workflow_definitions, approvals	Cross-cutting orchestration
Support	Tickets, comments, SLA handling	tickets, ticket_comments	Customer-facing service module
Reporting	Aggregated analytics and exports	materialized views, snapshots, exports	Read optimized, no ownership of transactional truth
Audit	Immutable activity and security logging	audit_logs, login_events	Appends from all domains

6. Separate super admin project

The super admin capability should not be merged into the tenant application UI. It should be delivered as a separate React project with its own routes, permissions, navigation, branding treatment, deployment pipeline, and audit policy. It may reuse a shared component library and selected backend packages, but its runtime boundary should remain separate.

Why it should be separate

Reduces the blast radius of privileged features and lowers the chance of accidental cross-tenant exposure.

Makes RBAC cleaner: platform administrators, support operations, finance operations, and compliance roles are

not mixed with tenant roles.

Allows independent release cadence, stronger audit logging, and separate MFA or network controls for highly privileged users.

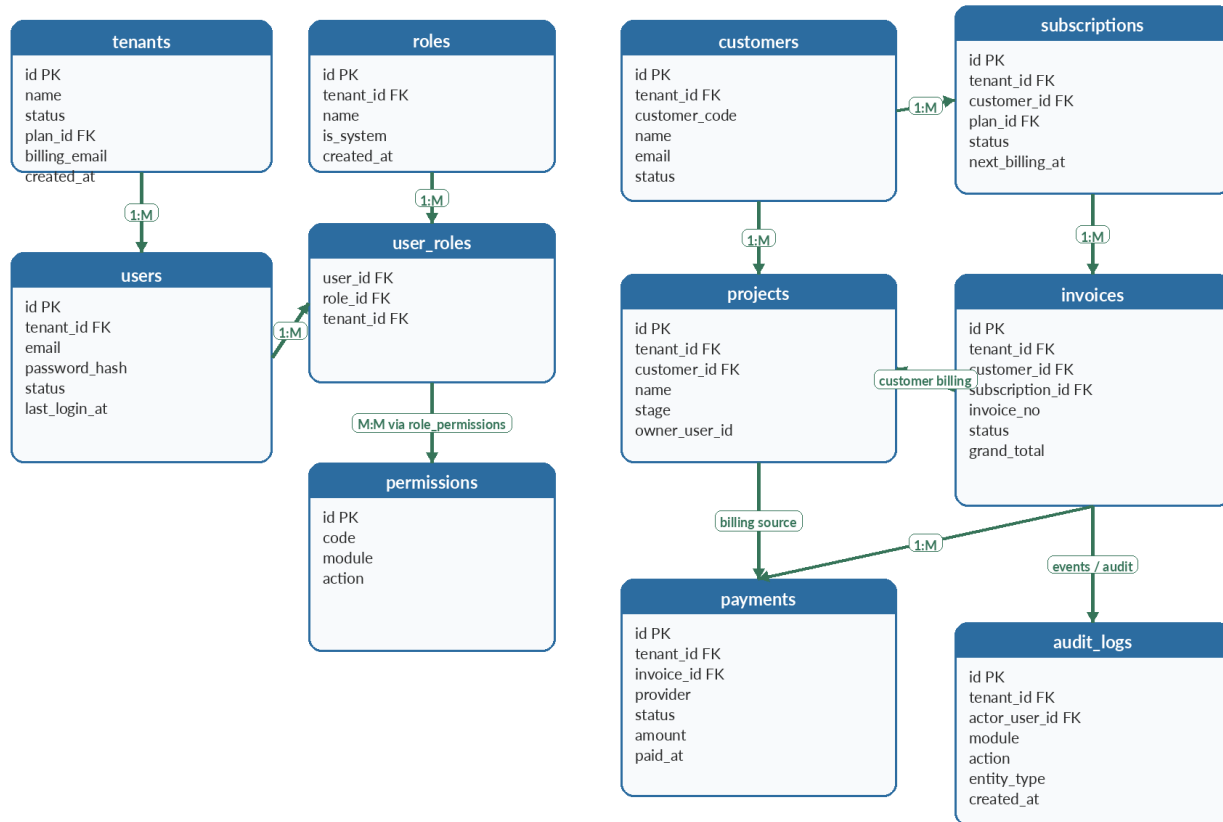
Keeps tenant UX focused on operational work instead of platform-management concerns.

Area	Responsibilities	Guardrails
Tenant lifecycle	Create tenant, suspend, reactivate, archive, assign plan, seed defaults	Every action logged with justification and actor identity
Platform subscriptions	Global pricing plans, billing oversight, invoice disputes	No direct edits to tenant transactional records without approved override flow
Feature governance	Enable/disable modules, beta flags, regional controls	Use configuration records, not code edits in production
Support tooling	Safe impersonation or support session approval	Time-boxed, approved, visible to audit, masked where required
Compliance & risk	Access reviews, export approvals, retention policy triggers	Restricted to dedicated admin roles and dual control for sensitive actions
Operations dashboard	Health, alerts, queue depth, tenant usage trends	Read-only operational dashboards separate from write actions

7. Data architecture and database chart

PostgreSQL is the transactional system of record. The recommended starting point is one PostgreSQL cluster with clearly separated schemas or bounded domain tables, enforced tenant scoping, migration-based schema management, and a documented path for extracting heavy modules later. Every tenant-owned table must include `tenant_id`, `created_at`, `updated_at`, `created_by`, `updated_by`, and a soft-delete status when business retention rules require it.

Logical Database Chart - Core Tenant ERP Schema



Implementation rule: every tenant-owned table carries tenant_id, standard audit fields, soft-delete markers, and indexed foreign keys. Super-admin data stays in a separate admin database/project boundary.

Figure 2. Core logical database chart for the tenant ERP schema. This is a planning baseline, not the final exhaustive schema.

7.1 Core schema standards

- Use UUID primary keys for internal identity and separate human-friendly codes for invoice numbers, customer codes, ticket codes, and project numbers.
- Apply NOT NULL constraints intentionally, use check constraints for enum-like status values where appropriate, and keep reference data normalized.
- Create compound indexes around tenant_id plus the main filter columns, for example (tenant_id, status), (tenant_id, created_at), and unique (tenant_id, invoice_no).
- Prefer append-only records for financial history and audit events; do not overwrite material financial facts after posting.
- Use migration tooling for all schema changes, including backfill scripts, rollbacks, and release notes.

7.2 Proposed key tables and fields

Platform / identity

Table	Key fields
tenants	id, name, slug, status, plan_id, default_currency, timezone, billing_email, created_at
tenant_settings	id, tenant_id, key, value_json, updated_at

Table	Key fields
users	id, tenant_id, email, password_hash, full_name, status, last_login_at, created_at
roles	id, tenant_id, name, description, is_system, created_at
permissions	id, code, module, action, description
user_roles	tenant_id, user_id, role_id, assigned_at, assigned_by
sessions	id, user_id, tenant_id, refresh_token_hash, ip_address, user_agent, expires_at

CRM / sales / project

Table	Key fields
customers	id, tenant_id, customer_code, name, email, phone, status, tax_id, created_at
contacts	id, tenant_id, customer_id, name, email, phone, role, is_primary
leads	id, tenant_id, source, owner_user_id, stage, expected_value, created_at
opportunities	id, tenant_id, customer_id, stage, amount, close_date, owner_user_id
projects	id, tenant_id, customer_id, name, stage, start_date, end_date, owner_user_id
project_milestones	id, tenant_id, project_id, name, due_date, billing_status, amount

Billing / subscription / payment

Table	Key fields
subscription_plans	id, code, name, billing_cycle, price, currency, active
subscriptions	id, tenant_id, customer_id, plan_id, status, start_at, next_billing_at, cancel_at
invoices	id, tenant_id, customer_id, subscription_id, invoice_no, status, subtotal, tax_total, grand_total, issued_at, due_at
invoice_items	id, tenant_id, invoice_id, line_no, item_type, description, qty, unit_price, tax_rate, line_total
payments	id, tenant_id, invoice_id, provider, provider_ref, status,

Table	Key fields
	amount, currency, paid_at
refunds	id, tenant_id, payment_id, amount, reason, status, approved_by, created_at

Finance / workflow / support / audit

Table	Key fields
accounts	id, tenant_id, account_code, account_name, account_type, status
ledger_entries	id, tenant_id, journal_id, account_id, debit, credit, reference_type, reference_id, posted_at
workflow_definitions	id, tenant_id, code, name, trigger_event, status, definition_json
approvals	id, tenant_id, workflow_instance_id, approver_user_id, decision, comments, decided_at
tickets	id, tenant_id, customer_id, subject, priority, status, assigned_user_id, opened_at
audit_logs	id, tenant_id, actor_user_id, module, action, entity_type, entity_id, metadata_json, created_at

7.3 Data isolation rules

- Tenant data queries always filter by tenant_id in application code and database access abstractions.
- Super admin reads across tenants only through a separate admin path with explicit audit logging and approval where required.
- Direct cross-module database access is prohibited; modules communicate through services or events.
- Sensitive fields such as tax identifiers, secrets, and payment provider references should be encrypted or tokenized according to risk level.
- Soft delete is the default unless legal or financial retention rules require immutable archival instead.

8. Security, compliance, and operational best practices

Area	Minimum standard	Implementation notes
Authentication	JWT access token + rotating refresh token + MFA for privileged users	Store refresh token hashes only, track device/session, revoke on logout or compromise
Authorization	Role-based access control with explicit permissions	Permission naming pattern module.action; audit all privilege changes

Area	Minimum standard	Implementation notes
API security	TLS everywhere, rate limiting, input validation, safe serialization	Apply helmet, CORS policy, request size limits, and webhook signature verification
Data protection	Encryption in transit and at rest for sensitive records	Use secret manager, backup encryption, signed URLs for documents
Auditability	Immutable security and business audit logs	Log actor, tenant, action, entity, before/after summary, timestamp, IP when relevant
Operational resilience	Automated backups, restore tests, health checks, alerting	Define RPO/RTO targets and test them quarterly
Release control	CI/CD with mandatory review and automated gates	No manual database edits in production outside approved break-glass process

9. DevOps and deployment model

- Repository strategy: either one mono-repo with apps/tenant-web, apps/admin-web, apps/api, packages/shared-ui, packages/config, packages/types; or two repos if organizational controls require stricter separation.
- Environment strategy: local, development, QA, staging, production. Keep infrastructure, secrets, and environment variables separate.
- Deployment baseline: Docker images, reverse proxy / ingress, PostgreSQL managed service or HA setup, Redis, object storage, centralized logs, metrics, and tracing.
- CI pipeline: install, lint, type-check, unit tests, integration tests, build, security scans, migration check, artifact publish.
- CD pipeline: deploy to staging automatically, run smoke tests, require approval for production, enable rollback and previous image redeploy.

10. Testing and quality strategy

Test type	Scope	Tools	Release rule
Unit tests	Core services, validators, utility functions	Vitest/Jest	Required on changed modules
API integration tests	Routes, auth, tenant isolation, posting rules	Supertest + test DB	Must pass before merge
UI tests	Critical user journeys	Playwright	Run on staging and before production release
Contract tests	Gateway webhooks, admin APIs, exports	Mock servers / pact style tests	Required for third-party integrations
Performance tests	Invoice posting, payment webhook burst, exports	k6 / Artillery	Baseline before go-live and each major release

Test type	Scope	Tools	Release rule
Security tests	Dependency scans, SAST, auth checks, penetration testing	CI scanners + external review	Critical findings block release

11. Team structure and ownership

- Product owner / business analyst: requirement definition, prioritization, UAT sign-off, change control.
- Solution architect / technical lead: architecture governance, domain boundaries, API contracts, database review, security design.
- Frontend engineers: tenant app and admin app delivery, design system, accessibility, test automation.
- Backend engineers: domain modules, APIs, migrations, integrations, events, performance tuning.
- QA engineer: test strategy, automated regression, non-functional validation, release gate support.
- DevOps engineer: environments, CI/CD, infrastructure, observability, backup and recovery.
- Security/compliance reviewer: privileged access policy, audit, vulnerability management, release risk review.

12. Risks and mitigation plan

Risk	Impact	Mitigation
Unclear ERP scope or frequent requirement changes	Delays, rework, unstable delivery plan	Freeze MVP per phase, maintain change log, use ADRs and backlog prioritization discipline
Cross-tenant leakage through poorly designed queries	Critical security incident	Mandatory tenant context middleware, repository patterns, integration tests for isolation, admin-path segregation
Billing logic changes late in project	Financial defects and audit pain	Domain modeling workshops early, invoice lifecycle locked, idempotent integration design
Over-engineering into microservices too early	Slower delivery and ops burden	Start with modular monolith and extract only proven hotspots
Privileged admin features mixed with tenant app	Governance confusion and higher breach risk	Separate super admin project with distinct auth policy and audit logs
Weak migration and release discipline	Production instability and data drift	Migration reviews, staging rehearsal, rollback plans, tested restores

13. Recommended coding and engineering standards

- Adopt trunk-based or short-lived branch workflow with pull requests, code review, and conventional commits or release tags.
- Protect main branch with required checks: lint, type-check, tests, secret scan, dependency scan, and migration validation.

- Define naming conventions for modules, APIs, events, permissions, DB tables, columns, and job queues early and document them.
- Use DTO validation on every external input, sanitize outputs, and never trust client-supplied tenant identifiers.
- Implement structured logging from the beginning with correlation ID, actor ID, tenant ID, module, and request latency.
- Track architecture decisions in ADRs and review any changes to auth, tenancy, billing, integration, or data retention design.
- Write documentation alongside code: setup guide, API docs, runbooks, migration notes, and troubleshooting guides.

14. Suggested initial backlog

1. Set up repositories, package management, linting, formatting, TypeScript config, commit hooks, and CI pipeline.
2. Create database migration framework, base schema, audit columns, tenant context, and seed data pipeline.
3. Implement auth module with login, refresh, logout, password reset, RBAC, session tracking, and permission middleware.
4. Build tenant app shell, navigation, dashboard, generic data table, form primitives, and notification patterns.
5. Build super admin shell, tenant list, tenant detail, plan assignment, suspend/reactivate flow, and audit view.
6. Deliver customers, contacts, projects, and document upload foundations before complex finance workflows.
7. Implement subscription plans, invoice generation, payment webhooks, posting rules, and financial reports in controlled order.

15. Final recommendation

Proceed with React, Express, and PostgreSQL as the approved stack, but execute the solution with TypeScript, modular architecture, disciplined schema design, and strong operational controls. Build the tenant ERP and super admin as separate web projects from the start. Use a modular monolith first, not a premature microservice landscape. Lock down tenant isolation and billing lifecycle rules early. Treat auditability, backups, migrations, and testing as release-blocking concerns rather than optional cleanup.

Go-live readiness checklist

Tenant isolation tested across API, UI, exports, reports, and admin paths.

Invoice, payment, refund, and subscription lifecycles validated with real edge cases.

Backups restored successfully in a rehearsal environment.

Monitoring, alerts, runbooks, and escalation matrix reviewed by the team.

Super admin MFA, privileged action logging, and support access controls verified.